



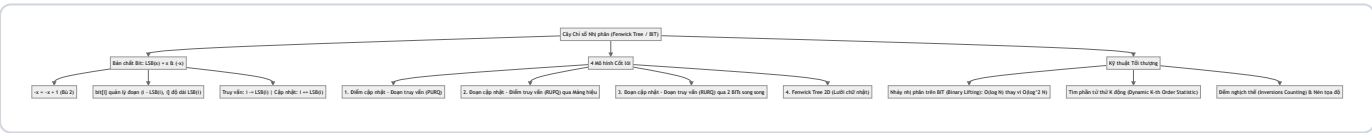
Chuyên đề: Cây Chỉ số Nhị phân (Fenwick Tree / Binary Indexed Tree - BIT)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Tổng quan & Bản đồ Tư duy (Mindmap)

Cây Chỉ số Nhị phân (Binary Indexed Tree - BIT), thường được gọi là **Cây Fenwick (Fenwick Tree)**, được nhà khoa học máy tính Peter M. Fenwick đề xuất vào năm 1994. Trong lập trình thi đấu (Competitive Programming), Fenwick Tree là cấu trúc dữ liệu được yêu thích hàng đầu nhờ:

- Mã nguồn siêu ngắn gọn:** Chỉ cần khoảng 5 đến 10 dòng code cho cả cập nhật và truy vấn.
- Tốc độ thực thi vượt trội (Cache-Friendly):** Sử dụng mảng phẳng 1 chiều liên tục, không có con trỏ, không có đệ quy, chạy nhanh hơn **Cây phân đoạn (Segment Tree)** từ **3 đến 5 lần**.
- Bộ nhớ tối ưu tuyệt đối:** Chỉ tiêu tốn đúng N phần tử bộ nhớ (thay vì $4N$ như Segment Tree).



2. Bản chất Toán học & Phép toán Bitwise $LSB(x)$

2.1. Phép toán $LSB(x) = x \& (-x)$

Trong hệ thống số nhị phân biểu diễn số nguyên có dấu theo chuẩn **Bù 2 (Two's Complement)**:

$$-x = (\sim x) + 1$$

Khi thực hiện phép toán bitwise AND giữa x và $-x$:

$$LSB(x) = x \& (-x)$$

kết quả thu được là một lũy thừa của 2 (2^k), tương ứng với **bit 1 có trọng số nhỏ nhất (Least Significant Bit - LSB)** của x .

Ví dụ:

$$-x = 12 = 1100_2 \implies -12 = \dots 0100_2 \implies 12 \& (-12) = 0100_2 = 4.$$

$$-x = 7 = 0111_2 \implies -7 = \dots 1001_2 \implies 7 \& (-7) = 0001_2 = 1.$$

2.2. Đoạn Quản lý của từng Nút

Khác với Segment Tree chia đôi đoạn liên tục, Fenwick Tree gán cho mỗi chỉ số $i \in [1, N]$ trách nhiệm quản lý nửa đoạn kết thúc tại i có độ dài đúng bằng $\text{LSB}(i)$:

$$\text{Đoạn quản lý của nút } i : (i - \text{LSB}(i), i] = [i - \text{LSB}(i) + 1, i]$$

Chỉ số i	Dạng nhị phân	$\text{LSB}(i)$	Đoạn quản lý của $\text{bit}[i]$	Độ dài đoạn
1	0001 ₂	1	[1, 1]	1
2	0010 ₂	2	[1, 2] (= [2 - 2 + 1, 2])	2
3	0011 ₂	1	[3, 3]	1
4	0100 ₂	4	[1, 4] (= [4 - 4 + 1, 4])	4
5	0101 ₂	1	[5, 5]	1
6	0110 ₂	2	[5, 6] (= [6 - 2 + 1, 6])	2
7	0111 ₂	1	[7, 7]	1
8	1000 ₂	8	[1, 8] (= [8 - 8 + 1, 8])	8

Nhận xét: Mọi số nguyên dương p đều có thể phân tích duy nhất thành tổng các lũy thừa của 2 rời nhau (chính là các bit 1 trong biểu diễn nhị phân của p). Do đó, đoạn tiền tố $[1, p]$ được ghép nối hoàn hảo bởi tối đa $\lfloor \log_2 p \rfloor + 1$ đoạn con không giao nhau!

2.3. Hai Thao tác Cơ bản

1. Truy vấn Tiền tố (Prefix Sum Query): $i -= i \& -i$

Để tính tổng các phần tử từ 1 đến p : Ta bắt đầu từ p , cộng giá trị $\text{bit}[p]$, sau đó nhảy ngược về trước bằng cách triệt tiêu bit 1 cuối cùng ($p -= p \& -p$).

```
long long query(int p) {
    long long sum = 0;
    for (; p > 0; p -= p & -p) {
        sum += bit[p];
    }
    return sum;
}
```

Tính tổng đoạn $[L, R]$: $prefix(R) - prefix(L - 1)$.

2. Cập nhật Điểm (Point Update): `i += i & -i`

Khi giá trị tại vị trí p tăng thêm một lượng `val`: Ta phải cập nhật tất cả các nút `bit[k]` mà đoạn quản lý của nó bao chứa p . Các nút cha này được xác định tuần tự bằng cách cộng dồn LSB (`p += p & -p`).

```
void update(int p, long long val) {
    for (; p <= n; p += p & -p) {
        bit[p] += val;
    }
}
```

Ghi chú:

Cả hai vòng lặp đều duyệt qua các bit 1 của số nguyên, do đó số bước lặp tối đa không bao giờ vượt quá $\lfloor \log_2 N \rfloor$ bước $\implies$ Thời gian chạy $O(\log N)$ tuyệt đối**.

3. Bốn Cấp độ Nâng cao của Fenwick Tree

3.1. Cấp độ 1: Cập nhật Điểm — Truy vấn Đoạn (Point Update, Range Query - PURQ)

Đây là mô hình nguyên bản của Fenwick Tree:

- Cập nhật $A[p] \leftarrow A[p] + V$: `update(p, v)` trong $O(\log N)$.
- Truy vấn tổng $\sum_{i=L}^R A[i]$: `query(R) - query(L - 1)` trong $O(\log N)$.

3.2. Cấp độ 2: Cập nhật Đoạn — Truy vấn Điểm (Range Update, Point Query - RUPQ)

Xét bài toán: Thêm V vào tất cả phần tử trong đoạn $[L, R]$, và truy vấn giá trị của phần tử tại vị trí p .

- **Kỹ thuật Mảng hiệu (Difference Array)**: Định nghĩa mảng $D[i] = A[i] - A[i - 1]$ (với quy ước $A[0] = 0$).
- Khi đó:

$$A[p] = \sum_{i=1}^p D[i] = prefix_D(p)$$

- Thao tác cộng V vào đoạn $[L, R]$ chỉ làm biến đổi đúng 2 vị trí trên mảng hiệu:

$$D[L] \leftarrow D[L] + V, \quad D[R + 1] \leftarrow D[R + 1] - V$$

- **Cài đặt**: Dựng cây Fenwick quản lý mảng hiệu D :
- Cập nhật đoạn $[L, R]$: `update(L, v)` và `update(R + 1, -v)`.
- Truy vấn giá trị tại p : `query(p)`.

3.3. Cấp độ 3: Cập nhật Đoạn — Truy vấn Đoạn (Range Update, Range Query - RURQ)

Làm thế nào để hỗ trợ **cả cập nhật đoạn và tính tổng đoạn** trong $O(\log N)$ chỉ bằng Fenwick Tree mà không cần Segment Tree Lazy phức tạp?

Biến đổi Đại số

Xét tổng tiền tố của mảng A :

$$\sum_{i=1}^p A[i] = \sum_{i=1}^p \sum_{j=1}^i D[j]$$

Thay đổi thứ tự lấy tổng: Mỗi phần tử $D[j]$ ($1 \leq j \leq p$) sẽ xuất hiện trong các tổng con từ $i = j$ đến $i = p$, tức xuất hiện đúng $(p - j + 1)$ lần:

$$\sum_{i=1}^p A[i] = \sum_{j=1}^p (p - j + 1) \cdot D[j] = (p + 1) \sum_{j=1}^p D[j] - \sum_{j=1}^p j \cdot D[j]$$

Giải pháp 2 Cây Fenwick Song song

Ta duy trì **2 cây Fenwick độc lập**:

- Cây B_1 quản lý mảng hiệu: $D[j]$
- Cây B_2 quản lý mảng trọng số: $j \cdot D[j]$

1. **Khi cộng V vào đoạn $[L, R]$:**
2. Trên B_1 : `update(L, V)` và `update(R + 1, -V)`.
3. Trên B_2 : `update(L, L * V)` và `update(R + 1, -(R + 1) * V)`.
4. **Khi truy vấn tổng tiền tố $[1, p]$:**

$$prefix(p) = (p + 1) \cdot B_1.query(p) - B_2.query(p)$$

5. **Tổng đoạn $[L, R]$:** $prefix(R) - prefix(L - 1)$.

💡 Mẹo / Tối ưu:

Kỹ thuật này giải quyết trọn vẹn bài toán cập nhật đoạn và truy vấn tổng đoạn trong $O(\log N)$ thời gian**, code chỉ tốn 25 dòng và chạy nhanh gấp 4 lần Segment Tree Lazy Propagation!

3.4. Cấp độ 4: Nhảy Nhị phân trên Fenwick Tree (Binary Lifting on BIT) — $O(\log N)$

Bài toán: Cho mảng các số nguyên không âm (hoặc mảng tần số). Tìm vị trí p nhỏ nhất sao cho tổng tiền tố:

$$prefix(p) \geq X$$

(Thao tác cốt lõi để tìm phần tử thứ K trong Cây thống kê thứ tự động - Order Statistic Tree).

- **Cách tiếp cận thông thường:** Chặt nhị phân vị trí $p \in [1, N]$. Mỗi bước gọi `query(mid)` $\implies$ Độ phức tạp $O(\log^2 N)$.
- **Kỹ thuật Nhảy nhị phân trực tiếp trên cây (Binary Lifting):**
- Tương tự như Binary Lifting trên cây hoặc Sparse Table, ta duyệt các lũy thừa của 2 từ lớn về bé: $2^{\lfloor \log_2 N \rfloor}, \dots, 2^1, 2^0$.
- Nhận xét then chốt: Nút `bit[pos + (1 << i)]` chứa **đúng tổng của khối kích thước 2^i** tiếp theo!
- Do đó, ta có thể so sánh trực tiếp X với `bit[pos + (1 << i)]` trong $O(1)$ mà không cần gọi hàm `query` !

```
int lower_bound(long long target) {
    long long sum = 0;
    int pos = 0;
    // Bắt đầu từ lũy thừa lớn nhất của 2 <= n
    for (int i = 1 << (31 - __builtin_clz(n)); i > 0; i >>= 1) {
        if (pos + i <= n && sum + bit[pos + i] < target) {
            sum += bit[pos + i];
            pos += i; // Nhảy bước dài 2^i
        }
    }
    return pos + 1; // Vị trí đầu tiên có prefix sum >= target
}
```

✦ Quan trọng:

Thuật toán trên giảm độ phức tạp tìm kiếm từ $O(\log^2 N)$ xuống **chính xác $O(\log N)$** , chỉ mất khoảng 18-20 phép tính bit cho $N = 2 \cdot 10^5$!

4. Fenwick Tree 2 Chiều (2D Fenwick Tree)

Trên lưới 2D kích thước $N \times M$:

- Mỗi ô (x, y) quản lý một hình chữ nhật con có kích thước $\text{LSB}(x) \times \text{LSB}(y)$.
- Cập nhật điểm (x, y) cộng thêm V : Hai vòng lặp lồng nhau duyệt theo LSB trong $O(\log N \log M)$.
- Truy vấn tổng hình chữ nhật $[x_1, y_1] \rightarrow [x_2, y_2]$ qua **Nguyên lý Bù trừ (Inclusion-Exclusion Principle)**:

$$\text{Sum} = S(x_2, y_2) - S(x_1 - 1, y_2) - S(x_2, y_1 - 1) + S(x_1 - 1, y_1 - 1)$$

```

void update2D(int x, int y, long long val) {
    for (int i = x; i <= n; i += i & -i) {
        for (int j = y; j <= m; j += j & -j) {
            bit2d[i][j] += val;
        }
    }
}

long long query2D(int x, int y) {
    long long sum = 0;
    for (int i = x; i > 0; i -= i & -i) {
        for (int j = y; j > 0; j -= j & -j) {
            sum += bit2d[i][j];
        }
    }
    return sum;
}

```

5. Bảng Trace Chi tiết: Cập nhật & Truy vấn trên Dãy 8 phần tử

Giả sử ban đầu ta có mảng $A = [0, 0, 0, 0, 0, 0, 0, 0]$ ($N = 8$).

Thực hiện:

1. `update(3, 5)` (Thêm 5 vào vị trí 3).
2. `update(6, 7)` (Thêm 7 vào vị trí 6).

5.1. Trace Thao tác `update(3, 5)`

- Bắt đầu: $p = 3 = 0011_2 \implies \text{LSB}(3) = 1$. Cộng vào `bit[3]`: `bit[3] += 5`.
- Tiếp theo: $p = 3 + 1 = 4 = 0100_2 \implies \text{LSB}(4) = 4$. Cộng vào `bit[4]`: `bit[4] += 5`.
- Tiếp theo: $p = 4 + 4 = 8 = 1000_2 \implies \text{LSB}(8) = 8$. Cộng vào `bit[8]`: `bit[8] += 5`.
- Tiếp theo: $p = 8 + 8 = 16 > 8 \implies$ Dừng!
 $\implies$ Các nút được cập nhật: `bit[3]`, `bit[4]`, `bit[8]`.

5.2. Trace Thao tác `update(6, 7)`

- Bắt đầu: $p = 6 = 0110_2 \implies \text{LSB}(6) = 2$. Cộng vào `bit[6]`: `bit[6] += 7`.
- Tiếp theo: $p = 6 + 2 = 8 = 1000_2 \implies \text{LSB}(8) = 8$. Cộng vào `bit[8]`: `bit[8] += 7`.
- Tiếp theo: $p = 8 + 8 = 16 > 8 \implies$ Dừng!
 $\implies$ Các nút được cập nhật: `bit[6]`, `bit[8]` (`bit[8]` hiện có giá trị $5 + 7 = 12$).

5.3. Trace Thao tác `query(7)` (Tính tổng $A[1 \dots 7]$)

- Bắt đầu: $p = 7 = 0111_2 \implies \text{LSB}(7) = 1$. Cộng `bit[7]` (giá trị 0).
- Nhảy: $p = 7 - 1 = 6 = 0110_2 \implies \text{LSB}(6) = 2$. Cộng `bit[6]` (giá trị 7).
- Nhảy: $p = 6 - 2 = 4 = 0100_2 \implies \text{LSB}(4) = 4$. Cộng `bit[4]` (giá trị 5).
- Nhảy: $p = 4 - 4 = 0 \implies$ Dừng!
 $\implies$ Tổng thu được: $0 + 7 + 5 = 12$. Khớp hoàn toàn với giá trị thực tế $A[3] + A[6] = 5 + 7 =$

⚠ 6. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu

1. **Vòng lặp Vô hạn khi Chỉ số $i = 0$:**
2. Nếu $p = 0$, ta có $\text{LSB}(0) = 0$ & $(-0) = 0$. Vòng lặp `p += p & -p` sẽ trở thành `p += 0`, dẫn đến **Infinite Loop (Treo máy / TLE)**!
3. **Quy tắc bắt buộc:** Fenwick Tree **BẮT BUỘC phải là 1-indexed** (các chỉ số phải từ 1 đến N). Nếu đề bài cho chỉ số từ 0, luôn tăng chỉ số lên 1 (`idx + 1`).
4. **Tràn số với Phép nhân $j \cdot D[j]$ trong Cấp độ 3 (RURQ):**
5. Khi thực hiện `update(L, L * V)`, với $L \approx 2 \cdot 10^5$ và $V \approx 10^9$, tích $L \cdot V \approx 2 \cdot 10^{14}$ sẽ gây tràn kiểu `int` 32-bit! Luôn ép kiểu `1LL * L * V` và sử dụng `long long` cho toàn bộ mảng cây Fenwick.
6. **Bài toán có Giá trị $A[i]$ Lớn ($A[i] \leq 10^9$):**
7. Khi sử dụng Fenwick Tree để đếm số nghịch thế hoặc đếm tần số, không thể khai báo mảng kích thước 10^9 .
8. **Bắt buộc kết hợp Kỹ thuật Rời rạc hóa / Nén tọa độ (Coordinate Compression):** Dùng `std::sort` và `std::unique` để nén các giá trị ban đầu về tập $[1, N]$ trước khi thao tác trên BIT.
9. **Không khởi tạo lại Cây khi có Nhiều Bộ test (Multi-test Cases):**
10. Với T bộ test, việc gọi `memset(bit, 0, sizeof(bit))` trên mảng lớn tối đa có thể gây **TLE**. Chỉ nên `fill(bit.begin(), bit.begin() + n + 2, 0)` theo đúng kích thước N của từng test case.

📚 7. Tài liệu Tham khảo & Đọc thêm

1. **CP-Algorithms:**
2. [Fenwick Tree \(Binary Indexed Tree\)](#) — Cập nhật điểm, truy vấn đoạn và Fenwick 2D.
3. **VNOI Wiki:**
4. [Cây chỉ số nhị phân](#) — Phân tích bitwise LSB và ứng dụng đếm nghịch thế.
5. [Fenwick Tree 2D](#) — Quản lý tổng vùng chữ nhật trên lưới.
6. **TopCoder Educational:**
7. [Binary Indexed Trees Tutorial](#) — Kỹ thuật Binary Lifting tìm phần tử thứ K trên Fenwick.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agent AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.